

Group 9 - Hostel Room Booking and Fees Management System

#	Full name	Registration number
1	Noah KUSAASIRA	VU-BSF-2603-3572-EVE
2	Moses Kyegombe	VU-BCD-2603-0677-EVE
3	Fardin Khan	VU-BSF-2603-1958-EVE
4	Asimire Leonard	VU-BSF-2603-0377-EVE

1. Overview

The Hostel Room Booking and Fees Management System is a menu-driven Python console application designed to replace the hostel warden's paper ledger. It allows the warden to allocate students to rooms, record fee payments, search for students, generate occupancy and defaulter reports, and have all data stored automatically between sessions all through a simple numbered menu that requires no programming knowledge to operate.

2. System Structure

The system is organised around one core class, `HostelSystem`, which holds two in-memory data structures:

- Blocks - a nested dictionary of the form `{block_name: {room_name: {"capacity": int, "occupants": [reg_no,]}}`, covering three predefined hostel blocks, BLOCK-A, BLOCK-B, and BLOCK-C, each with its own fixed set of rooms (ROOM1, ROOM2,) and a maximum bed capacity per room.
- Students - a dictionary keyed by registration number, each entry holding the student's name, assigned block/room, total fee, amount paid, outstanding balance, and a full payment history (amount and timestamp for every transaction).

Every operation, allocation, payments, search, reporting, saving/loading is implemented as a documented method on this class (each carries its own docstring), keeping the business logic separate from the menu functions that drive it.

3. Predefined Hostel Data

Block	Rooms	Capacities
BLOCK-A	ROOM1	2
	ROOM2	2
	ROOM3	3
BLOCK-B	ROOM1	2
	ROOM2	2

BLOCK-C	ROOM1	2
	ROOM2	2
	ROOM3	3
	ROOM3	4

This gives 9 rooms and 22 beds in total across the three blocks This form the default block data the system needs on first run or whenever the saved JSON data file is missing or unusable.

4. Key Features

Requirement	How it's implemented
Data setup	Three blocks are predefined with fixed, named rooms and per-room capacities; a startup summary shows beds occupied vs. total capacity per block and overall
Registration & allocation	A student is allocated to a room only if it has free capacity and the registration number isn't already in use; full rooms are rejected with a clear message (e.g. "Room ROOM1 in Block BLOCK-A is already full (2/2)"), and occupancy updates immediately on success
Fee payments	Full or partial payments are recorded against a student's account; the system accumulates total paid, recalculates the outstanding balance after every payment, and logs each payment with a full date-and-time stamp.
Search & reporting	Students can be found by exact registration number or partial name match. Per-block (or all-block) occupancy reports list every room and its occupants. A defaulters report lists students whose balance exceeds a warden-supplied threshold, sorted highest balance first.
Data storage	All blocks and student records are saved to hostel_data.json and reloaded automatically on startup. Missing, empty, corrupted, or wrongly structured JSON files are all detected and handled gracefully, the system starts with a fresh state and a warning rather than crashing.
Menu-driven driver	A single looping menu with options 1 to 8, ties every feature together, with input validation throughout: empty fields, non-numeric amounts, and negative or invalid numbers are all caught and re-prompted rather than allowed to crash the program

5. Data Storage Approach

Data is stored as human-readable JSON rather than a binary format, making it easy for a non-technical warden or a developer troubleshooting an issue to inspect the file directly if needed. Saving happens automatically after a successful allocation or payment and on exit, and is also available on demand from the menu option 7, so no work is lost if the session ends unexpectedly.

6. Error Handling

The `_load_data()` method anticipates several distinct failure modes rather than assuming the file is always well-formed:

- File missing - the system starts fresh and informs the warden.
- File empty or invalid JSON - caught via `json.JSONDecodeError`.
- Valid JSON but the wrong data structure for example a list instead of an object, or blocks/students not present as dictionaries - caught via an explicit `isinstance` check that raises a `ValueError`.
- Unreadable file (permissions, disk issues) - caught via `OSError`.

In every case the program prints a clear warning and continues with a fresh system rather than crashing, satisfying the requirement to handle a missing or damaged file gracefully.

7. Testing Performed

The system was tested end-to-end before delivery, covering:

- Successful room allocation and correct occupancy updates
- Rejection of allocation into a full room, with a clear explanation
- Full and partial fee payments accumulating correctly across multiple transactions for the same student
- Search by both registration number and partial name
- Block-level and all-block occupancy reports
- Fee defaulter filtering at a chosen threshold
- Reloading previously saved data on a fresh run
- Graceful recovery from a missing data file, an empty file, invalid JSON, and valid JSON with the wrong structure
- Rejection and re-prompting of invalid numeric input (non-numeric, negative, and zero values)

8. Limitations and Possible Extensions

- The system currently runs as a single-user console tool; it does not support concurrent access by multiple wardens at once.
- Room reassignment/checkout, moving or removing a student from a room is not yet implemented - this would be a natural next feature.
- There is no authentication layer, since the specifies a single warden operating the tool directly.

9. Conclusion

The delivered system fulfils all six requirements, predefined hostel data, validated room allocation, cumulative fee tracking, search and reporting tools, automatic and fault-tolerant file persistence, and a single cohesive menu that a non-technical warden can use confidently in place of the paper ledger.